ASSIGNMENT-GEN AI MASTERY

Name: Grande Praveen
Reg No: 99230040558
Dept: CSE (AIML)
Date: 05-09-2026

1. What is the advantage of using {user_input} instead of hard-coding input?

When we hard-code a value directly into a prompt, the prompt becomes usable for that one specific case only. Using a placeholder like {user_input} instead lets us write the prompt once and reuse it for any input at runtime, since the actual value gets substituted in later. This makes the prompt dynamic and reusable rather than static. It also keeps the prompt logic separate from the data, which is much easier to maintain, test, and scale, especially in an application where hundreds of different users are sending different inputs.

2. When would .format() be enough?

Python's .format() method works well for simple templates where we just need to plug in a few values into fixed text, with no real logic involved, no loops, and no conditions. For example, a short prompt like "Translate the following text to {language}: {text}" is a perfectly good case for .format(), since it's just straightforward substitution. It's fast, readable, and doesn't need any extra setup, so for small and simple prompts it's usually the most practical choice.

3. When might Jinja2 be useful?

Jinja2 becomes useful once the prompt needs more than simple substitution, for instance when it needs to include conditional sections (show this part only if a certain condition is true), loops (like listing multiple few-shot examples dynamically), or reusable template blocks. It is much better suited for complex, structured, or large prompts because it keeps the template readable even when there's branching logic involved, something .format() simply cannot handle cleanly.

4. Why should prompts be versioned?

Prompts are rarely perfect on the first attempt, they usually get refined over time. If we version our prompts, we can track exactly what changed between iterations, compare how each version performs, and roll back quickly if a new version turns out to perform worse. Versioning also helps when multiple people are working on the same project, since everyone can see the history

of changes and understand why a particular version was chosen. In short, it brings the same discipline to prompts that version control brings to code.

5. What is a prompt regression?

A prompt regression happens when a change made to improve a prompt ends up making it perform worse on certain cases that used to work fine before. So even if the new version looks better overall, it might have quietly broken something that was working correctly earlier. This is why testing a new prompt version against previous test cases is important, so regressions get caught before deployment instead of showing up later as unexpected failures.

6. Why use the same evaluation set when comparing prompt versions?

If we test different prompt versions on different inputs, we cannot really tell whether the difference in performance is because of the prompt change or simply because the inputs were different in difficulty. Using the same evaluation set for every version keeps the comparison fair, since the only thing changing is the prompt itself. This gives a much more reliable and consistent measure of whether a new version is genuinely better than the previous one.